🚀 Getting Started Guide: Pokémon Card Auction App

Project: Livestream Auction App for Collectible Cards
Your Experience Level: Some coding (can read/modify code)
IDE: Cursor
Tech Stack: React (web) → React Native (mobile later)

📑 Table of Contents

1. Setting Up Your Development Environment
2. Understanding Cursor IDE
3. Project File Structure
4. Git Basics & Version Control
5. Development Workflow
6. Cursor AI Prompts Cheat Sheet

1️⃣ Setting Up Your Development Environment

Prerequisites Installation

Step 1: Install Node.js (JavaScript runtime)

1. Go to https://nodejs.org/
2. Download the **LTS version** (Long Term Support) - currently v20.x
3. Run the installer and follow prompts
4. Verify installation:

```bash
node --version
# Should show: v20.x.x

npm --version
# Should show: 10.x.x
```

Step 2: Install Git (version control)

1. Go to https://git-scm.com/downloads
2. Download for your operating system
3. Run installer (use default settings)
4. Verify installation:

```bash
git --version
# Should show: git version 2.x.x
```

Step 3: Install Cursor

1. Go to https://cursor.com/
2. Download and install
3. Open Cursor - it will look very similar to VS Code if you've used it

Step 4: Configure Git (First Time Only)

Open Cursor's terminal (View → Terminal or Ctrl+`/ Cmd+`) and run:

```bash
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

This identifies you as the author of your code changes.

2️⃣ Understanding Cursor IDE

Interface Overview

```
┌──────────────────────────────────────────────────────────┐
│ File Explorer │ Code Editor        │ Cursor Chat  │
│               │                    │              │
│ 📁 frontend   │ Your code here     │ Ask AI for   │
│ 📁 backend    │                    │ help here    │
│ 📁 shared     │                    │              │
│ 📄 README.md  │                    │ 💬 Chat      │
│               │                    │ 🎼 Composer  │
└──────────────────────────────────────────────────────────┘
```

```
│          Terminal (bottom panel)          │
└──────────────────────────────────────────┘
```

Key Features You'll Use

1. Cursor Chat (Cmd+L / Ctrl+L)

- Ask questions about your code
- Get explanations of errors
- Request code snippets
- Example: "Explain what this function does"

2. Cursor Composer (Cmd+I / Ctrl+I)

- Make changes across multiple files at once
- Example: "Add authentication to all protected routes"
- Great for large features

3. Inline Edit (Cmd+K / Ctrl+K)

- Highlight code and ask AI to modify it
- Example: Select a function, hit Cmd+K, type "add error handling"

4. Terminal (Ctrl+`` / Cmd+``)

- Run commands
- Start your development server
- Execute Git commands

5. Autocomplete

- As you type, Cursor suggests completions
- Press Tab to accept suggestions
- It understands your entire codebase

Cursor Settings to Configure

1. **Enable Cursor Tab (Autocomplete)**
    - Settings → Features → Cursor Tab → Enable
2. **Choose AI Model**

- Settings → Models
- Recommended: Claude 3.5 Sonnet (best for coding)
- You get free usage credits to start

3. **Privacy Mode** (Optional)
    - Settings → Privacy
    - If working with sensitive data, enable Privacy Mode
    - For learning projects, standard mode is fine

3️⃣ Project File Structure

Complete Folder Structure

When you create your project, it will look like this:

```
pokemon-auction-app/
│
├── frontend/                # React web application
│   ├── public/
│   │   ├── index.html
│   │   └── favicon.ico
│   │
│   ├── src/
│   │   ├── components/        # Reusable UI components
│   │   │   ├── auth/
│   │   │   │   ├── LoginForm.jsx
│   │   │   │   └── SignupForm.jsx
│   │   │   ├── cards/
│   │   │   │   ├── CardGrid.jsx
│   │   │   │   ├── CardDetail.jsx
│   │   │   │   └── UploadCard.jsx
│   │   │   ├── livestream/
│   │   │   │   ├── StreamViewer.jsx
│   │   │   │   ├── StreamChat.jsx
│   │   │   │   └── BiddingPanel.jsx
│   │   │   └── common/
│   │   │       ├── Button.jsx
│   │   │       ├── Input.jsx
│   │   │       └── Modal.jsx
│   │   │
│   │   ├── pages/             # Full page components
│   │   │   ├── HomePage.jsx
│   │   │   ├── LoginPage.jsx
│   │   │   ├── ProfilePage.jsx
│   │   │   ├── LivestreamPage.jsx
│   │   │   └── CardDetailPage.jsx
│   │   │
│   │   ├── services/          # API calls & external services
│   │   │   ├── api.js         # Base API configuration
│   │   │   ├── authService.js   # Login/signup functions
│   │   │   ├── cardService.js   # Card CRUD operations
│   │   │   ├── streamService.js # Livestream functions
│   │   │   └── bidService.js    # Bidding functions
│   │   │
│   │   ├── hooks/             # Custom React hooks
│   │   │   ├── useAuth.js
│   │   │   ├── useWebSocket.js
│   │   │   └── useStream.js
```

```
│   │   │
│   │   ├── context/          # React Context (global state)
│   │   │   ├── AuthContext.jsx
│   │   │   └── StreamContext.jsx
│   │   │
│   │   ├── utils/            # Helper functions
│   │   │   ├── formatters.js    # Date, price formatting
│   │   │   ├── validators.js    # Form validation
│   │   │   └── constants.js     # App-wide constants
│   │   │
│   │   ├── styles/           # CSS/styling files
│   │   │   ├── index.css
│   │   │   └── tailwind.css
│   │   │
│   │   ├── App.jsx           # Main app component
│   │   └── index.js          # Entry point
│   │
│   ├── package.json          # Frontend dependencies
│   ├── .env                  # Environment variables (API keys)
│   └── README.md
│
├── backend/                  # Node.js API server
│   ├── src/
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   │   ├── auth.controller.js  # Route handlers
│   │   │   │   ├── auth.service.js     # Business logic
│   │   │   │   └── auth.routes.js      # Route definitions
│   │   │   ├── users/
│   │   │   │   ├── users.controller.js
│   │   │   │   ├── users.service.js
│   │   │   │   └── users.routes.js
│   │   │   ├── cards/
│   │   │   │   ├── cards.controller.js
│   │   │   │   ├── cards.service.js
│   │   │   │   └── cards.routes.js
│   │   │   ├── streams/
│   │   │   │   ├── streams.controller.js
│   │   │   │   ├── streams.service.js
│   │   │   │   └── streams.routes.js
│   │   │   └── bids/
│   │   │       ├── bids.controller.js
│   │   │       ├── bids.service.js
│   │   │       └── bids.routes.js
```

```
│   │   │
│   │   ├── database/
│   │   │   ├── models/         # Database schemas
│   │   │   │   ├── User.js
│   │   │   │   ├── Card.js
│   │   │   │   ├── Stream.js
│   │   │   │   └── Bid.js
│   │   │   └── migrations/     # Database version history
│   │   │       └── 001_initial_schema.sql
│   │   │
│   │   ├── middleware/         # Request interceptors
│   │   │   ├── auth.middleware.js    # Check if user logged in
│   │   │   ├── error.middleware.js   # Handle errors
│   │   │   └── logger.middleware.js  # Log requests
│   │   │
│   │   ├── websocket/          # Real-time functionality
│   │   │   ├── socketHandler.js
│   │   │   └── bidHandler.js
│   │   │
│   │   ├── utils/
│   │   │   ├── logger.js
│   │   │   └── validators.js
│   │   │
│   │   ├── config/             # Configuration
│   │   │   ├── database.js
│   │   │   ├── redis.js
│   │   │   └── stripe.js
│   │   │
│   │   ├── app.js              # Express app setup
│   │   └── server.js           # Server entry point
│   │
│   ├── package.json            # Backend dependencies
│   ├── .env                    # Secrets (database URL, API keys)
│   └── README.md
│
├── shared/                     # Code used by both frontend & backend
│   ├── types/                  # TypeScript types (optional)
│   ├── constants.js            # Shared constants
│   └── validators.js           # Shared validation logic
│
├── .gitignore                  # Files Git should ignore
├── README.md                   # Project documentation
└── package.json                # Root package.json (for scripts)
```

What Each File Type Does

File Type	Purpose	Example
`.jsx` / `.js`	JavaScript/React components	`LoginForm.jsx`
`.css`	Styling	`index.css`
`.json`	Configuration & data	`package.json`
`.env`	Secret keys (NOT committed to Git)	`DATABASE_URL=...`
`.md`	Documentation (Markdown)	`README.md`
`.sql`	Database queries	`001_initial_schema.sql`

Understanding the Architecture

Frontend (React)

- Runs in user's browser
- Shows UI, handles user interactions
- Makes API calls to backend
- URL: `http://localhost:3000`

Backend (Node.js + Express)

- Runs on a server
- Handles business logic, database access
- Provides API endpoints for frontend
- URL: `http://localhost:5000`

Database (PostgreSQL)

- Stores all data (users, cards, bids)
- Accessed only by backend
- URL: `postgresql://localhost:5432/pokemon_auction`

Flow Example: User Places a Bid

```
User clicks "Bid $50" button
  ↓
Frontend: Sends POST /api/bids { amount: 50, cardId: 123 }
  ↓
Backend: Validates bid, saves to database
  ↓
Backend: Broadcasts bid via WebSocket to all viewers
  ↓
Frontend: Updates UI to show new highest bid
```

4️⃣ Git Basics & Version Control

What is Git?

Git is like a **time machine for your code**. It:

- Tracks every change you make
- Lets you undo mistakes
- Allows collaboration without conflicts
- Backs up your work to the cloud (GitHub)

Core Concepts

Repository (Repo)

- A project folder tracked by Git
- Contains all your code + history of changes

Commit

- A saved snapshot of your code at a point in time
- Like hitting "Save Version" in Google Docs
- Each commit has a message describing what changed

Branch

- A parallel version of your code
- Lets you work on features without breaking the main code
- Default branch is usually called `main`

Remote

- A copy of your repo on GitHub
- Backs up your code in the cloud
- Enables collaboration

Visual: How Git Works

```
Your Computer                GitHub (Remote)
_____________                _______________

Working Directory
(unsaved changes)
    ↓
  git add
    ↓
Staging Area
(changes ready to commit)
    ↓
  git commit
    ↓
Local Repository      ←———→   Remote Repository
(saved history)       git push  (cloud backup)
                      git pull
```

Step-by-Step: Setting Up Git for Your Project

Step 1: Create a GitHub Account

1. Go to https://github.com
2. Sign up for free
3. Verify your email

Step 2: Create a New Repository on GitHub

1. Click the + icon → "New repository"
2. Name: `pokemon-auction-app`
3. Description: "Livestream auction app for Pokémon cards"
4. Keep it **Public** (or Private if you prefer)
5. Check "Add a README file"
6. Click "Create repository"

Step 3: Clone the Repository to Your Computer

1. On your repo page, click the green **Code** button
2. Copy the HTTPS URL (looks like: `https://github.com/yourusername/pokemon-auction-app.git`)
3. Open Cursor's terminal
4. Navigate to where you want the project:

```bash
cd ~/Documents  # Or wherever you keep projects
```

5. Clone the repo:

```bash
git clone https://github.com/yourusername/pokemon-auction-app.git
cd pokemon-auction-app
```

Step 4: Open Project in Cursor

1. In Cursor: File → Open Folder
2. Select the `pokemon-auction-app` folder
3. You're now ready to code!

Daily Git Workflow

Every time you code, follow this pattern:

```bash
```

```
# 1. Before starting work, get latest changes (if collaborating)
git pull

# 2. Create a new branch for your feature
git checkout -b feature/user-authentication
# Branch naming convention: feature/name, bugfix/name, etc.

# 3. Make your code changes in Cursor...
# (Edit files, add components, etc.)

# 4. Check what files you changed
git status
# Shows: modified files, new files, deleted files

# 5. Stage files for commit (add to "shopping cart")
git add .
# The . means "add all changed files"
# Or add specific files: git add frontend/src/components/LoginForm.jsx

# 6. Commit your changes with a descriptive message
git commit -m "Add user login form with validation"
# Message should be: short, clear, present tense

# 7. Push your branch to GitHub
git push origin feature/user-authentication

# 8. When feature is complete, merge back to main
git checkout main
git merge feature/user-authentication
git push origin main
```

Commit Message Best Practices

Good commit messages:

```bash
git commit -m "Add livestream viewer component"
git commit -m "Fix bug in bidding logic causing race conditions"
git commit -m "Update card upload form with image preview"
git commit -m "Refactor authentication service for better error handling"
```

Bad commit messages:

```bash
git commit -m "changes"
git commit -m "fixes"
git commit -m "asdf"
git commit -m "stuff"
```

Format: `<verb> <what you did>`

- Common verbs: Add, Fix, Update, Refactor, Remove, Implement

Understanding Branches & Merging

Why use branches?

- Keep `main` branch stable (always working)
- Experiment without breaking things
- Work on multiple features simultaneously

Branch Strategy for This Project:

```
main (always stable, deployed code)
  │
  ├── feature/authentication     (you're coding login)
  │
  ├── feature/livestream-viewer  (you're coding video player)
  │
  └── bugfix/bid-timing-issue    (fixing a bug)
```

Merging Example:

```bash
# You're done with authentication feature
git checkout main                # Switch to main branch
git merge feature/authentication     # Merge your feature into main
git push origin main             # Push merged code to GitHub

# Delete the feature branch (cleanup)
git branch -d feature/authentication
```

Handling Merge Conflicts (Don't Panic!)

What is a merge conflict? When Git can't automatically combine changes because you and someone else edited the same lines.

How it looks:

```javascript
<<<<<<< HEAD
const userName = "Alice";
=======
const userName = "Bob";
>>>>>>> feature/update-username
```

How to resolve:

1. Open the file in Cursor
2. Choose which version to keep (or combine them)
3. Delete the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`)
4. Save the file
5. Add and commit:

```bash
git add .
git commit -m "Resolve merge conflict in username"
```

Using Git in Cursor

Visual Git in Cursor:

1. **Source Control Panel** (left sidebar, branch icon)
   - Shows changed files
   - Click files to see diffs (what changed)
   - Stage files by clicking + icon
   - Commit with message box at top
2. **Git Commands via Terminal** (recommended for learning)
   - More control and understanding
   - See exactly what's happening

Essential Git Commands Cheat Sheet

Command	What It Does
`git status`	Show current changes
`git add <file>`	Stage specific file
`git add .`	Stage all changes
`git commit -m "message"`	Save a snapshot
`git push`	Upload to GitHub
`git pull`	Download from GitHub
`git checkout -b <branch>`	Create new branch
`git checkout <branch>`	Switch to branch
`git merge <branch>`	Merge branch into current
`git log`	View commit history
`git diff`	See unstaged changes
`git reset --hard`	⚠️ Discard all changes (dangerous!)

.gitignore File

What is it? A file that tells Git to ignore certain files/folders (don't track them).

Why?

- Secrets (API keys, passwords)
- Large files (videos, images)
- Generated files (build output)
- Dependencies (node_modules)

Your .gitignore file:

```
# Dependencies
node_modules/
package-lock.json

# Environment variables (secrets!)
.env
.env.local

# Build output
build/
dist/
.next/

# IDE files
.vscode/
.idea/
*.swp
*.swo

# OS files
.DS_Store
Thumbs.db

# Logs
logs/
*.log

# Temporary files
tmp/
temp/
```

⚠️ **CRITICAL: Never commit .env files!** They contain API keys and passwords. If you accidentally commit one:

1. Delete it from Git: `git rm --cached .env`
2. Immediately rotate/change all API keys
3. Add `.env` to `.gitignore`

5️⃣ Development Workflow

First-Time Project Setup

Step 1: Create Project Structure

```bash
# From your pokemon-auction-app folder
mkdir frontend backend shared

# Create initial files
touch README.md .gitignore
```

Step 2: Initialize Frontend (React)

```bash
cd frontend
npx create-react-app .
npm install
```

This creates a React app with all necessary files and dependencies.

Step 3: Initialize Backend (Node.js + Express)

```bash
cd ../backend
npm init -y
npm install express cors dotenv pg socket.io
npm install --save-dev nodemon
```

Creates `package.json` and installs essential packages.

Step 4: Set Up Environment Variables

Create `.env` files:

```bash
```

```
# frontend/.env
REACT_APP_API_URL=http://localhost:5000

# backend/.env
PORT=5000
DATABASE_URL=postgresql://localhost:5432/pokemon_auction
JWT_SECRET=your-secret-key-here-change-in-production
```

Step 5: Initial Commit

```bash
cd ..  # Back to root
git add .
git commit -m "Initial project setup with frontend and backend scaffolding"
git push origin main
```

Daily Development Cycle

Morning Routine:

```bash
# 1. Open project in Cursor
# File → Open Recent → pokemon-auction-app

# 2. Pull latest changes (if working with others)
git pull

# 3. Create feature branch
git checkout -b feature/todays-task

# 4. Start development servers (in separate terminals)
# Terminal 1:
cd frontend
npm start
# Opens http://localhost:3000

# Terminal 2:
cd backend
npm run dev
# Runs on http://localhost:5000
```

Coding Loop:

1. **Make code changes** in Cursor
2. **Save files** (Cmd+S / Ctrl+S)
3. **Check browser -** React auto-refreshes
4. **Test functionality**
5. **Repeat**

End of Session:

```bash
# 1. Check what changed
git status

# 2. Stage and commit
git add .
git commit -m "Implement card upload with image preview"

# 3. Push to GitHub
git push origin feature/todays-task

# 4. (Optional) Merge to main if feature is complete
git checkout main
git merge feature/todays-task
git push origin main
```

Testing Your App Locally

Frontend testing:

1. Open http://localhost:3000 in your browser
2. Click around, test features
3. Open browser console (F12) to see errors

Backend testing:

- Use Postman or Thunder Client (Cursor extension)
- Send API requests to http://localhost:5000
- Check responses and status codes

Database testing:

- Install TablePlus or pgAdmin
- Connect to your local PostgreSQL database
- View tables, run queries

6️⃣ Cursor AI Prompts Cheat Sheet

General Coding Prompts

Understanding Code:

```
"Explain what this function does line by line"
"What does this error mean and how do I fix it?"
"Why is this component re-rendering constantly?"
```

Generating Code:

```
"Create a React component for a card grid that displays Pokémon cards"
"Write a function to validate email addresses"
"Add error handling to this API call"
```

Refactoring:

```
"Refactor this component to use hooks instead of class syntax"
"Simplify this code while keeping the same functionality"
"Extract this logic into a reusable custom hook"
```

Phase-Specific Prompts

Phase 1: Authentication

```
"Set up Supabase authentication in this React app"
"Create login and signup forms with validation"
"Implement protected routes that require authentication"
"Add a logout button that clears the user session"
```

Phase 2: Card Management

"Create a form to upload card images with preview"
"Build a card grid component that displays cards in a responsive layout"
"Add pagination to the card list (20 cards per page)"
"Implement a search filter for cards by name and set"

Phase 3: Livestreaming

"Integrate Daily.co video streaming into this component"
"Create a livestream viewer with chat on the side"
"Add viewer count that updates in real-time"
"Handle reconnection when the stream drops"

Phase 4: Bidding

"Set up WebSocket connection for real-time bid updates"
"Create a bidding panel with current bid display and bid button"
"Implement anti-sniping logic (extend auction by 30s on last-minute bids)"
"Validate that new bid is higher than current bid plus minimum increment"

Phase 5: Payments

"Integrate Stripe payment form in React"
"Create an escrow system that holds funds until delivery confirmation"
"Add a disputes page where users can view and respond to disputes"

Debugging Prompts

"This component isn't rendering. What's wrong?"
"I'm getting a CORS error. How do I fix this?"
"My WebSocket connection keeps disconnecting. Help debug."
"The API call returns 401 Unauthorized. What am I missing?"

Best Practices When Using Cursor

1. **Be specific:** Instead of "fix this," say "fix the CORS error when calling the /api/cards endpoint"
2. **Provide context:** Use Cursor Composer for multi-file changes
3. **Review generated code:** Don't blindly copy-paste; understand what it does
4. **Iterate:** If the first response isn't perfect, refine your prompt
5. **Use inline edits:** Highlight code and use Cmd+K to modify specific sections

---

🎯 Next Steps: Ready to Code!

You now understand:

- ✅ How to set up your development environment
- ✅ How Cursor IDE works
- ✅ The file structure of your project
- ✅ How Git tracks and backs up your code
- ✅ Daily development workflow
- ✅ How to use Cursor AI effectively

Your Action Items Right Now:

[ ] Step 1: Install Node.js, Git, and Cursor (if not already done)
[ ] Step 2: Create GitHub account and repository
[ ] Step 3: Clone repository and open in Cursor
[ ] Step 4: Run the setup commands to create project structure
[ ] Step 5: Make your first commit and push to GitHub

Once you've completed these, you'll be ready to start building!

📚 Additional Resources

React Documentation: https://react.dev
Node.js Guides: https://nodejs.org/en/docs/
Git Handbook: https://guides.github.com/introduction/git-handbook/
Cursor Documentation: https://cursor.sh/docs

Questions? Ask me anything! I'm here to guide you through each step. 🚀